



FEB Racing

Autonomous sim-racing manual

- Part I** For everyone: what the platform is and the rules
- Part II** For a new member: from nothing to a leaderboard entry
- Part III** For the organiser: tracks, events, race day, releases

Formula Electric at Berkeley
September 2026, platform release v0.1.0

<https://github.com/Pranman1/feb-racing>
<https://pranman1.github.io/feb-racing/>

Contents

I	The platform	2
1	What it is	2
2	Rules	3
II	New member: from nothing to a leaderboard entry	3
3	Install (once, about 15 minutes)	4
4	Write your driver	4
5	Score yourself and post it	6
6	Race the house racer	6
7	Enter competitions	6
8	Learning path	6
9	If something goes wrong	7
III	Organiser	7
10	The organiser PC	7
11	Tracks	7
12	Events	8
13	Race day: time-attack	8
14	Finals: head-to-head	9
15	Between events	9
16	Releasing the simulator	9
17	A semester	10
18	Organiser checklist for the first run-through	10
A	Command reference	10
B	Repository layout	11

Part I

The platform

1 What it is

FEB Racing is a sim-racing programme for autonomous driving. Members write a ROS 2 driving stack for a 1:10 scale RoboRacer car, run it in the FEB Simulator on their own laptop, and post lap times to a public leaderboard. Twice a semester the organiser runs a competition on a secret track, with the same rules as the RoboRacer Sim Racing League.

The simulator is a fork of the AutoDRIVE Simulator (Unity). The car, its sensors and its physics are the league's RoboRacer model, unchanged, so a stack that works here works in the league too.

1.1 The pieces

Piece	What it does	Who touches it
FEB Simulator	Native app (Mac, Windows, Linux). Simulates the car on a track folder and talks to a bridge over a WebSocket on port 4567.	Everyone runs it, nobody edits it
Devkit container	Docker image <code>ghcr.io/pranman1/feb-devkit</code> : ROS 2 Humble plus the AutoDRIVE bridge that turns the WebSocket into ROS 2 topics. The member's code runs inside it.	Members' code runs inside it
<code>stack/</code>	The member's ROS 2 packages. Mounted into the container and built on every start.	Members
<code>tracks/</code>	Public tracks. PNG and YAML in, <code>track.json</code> out. Shipped inside the app.	Organiser adds, members use
<code>feb-sim</code>	The member launcher: <code>setup</code> , <code>run</code> , <code>practice</code> , <code>submit</code> , <code>shell</code> , <code>stop</code> .	Members
<code>feb-race</code>	The organiser harness: events, nightly reruns, verification, head-to-head brackets.	Organiser
<code>results/</code> , <code>events/</code> , <code>submissions.yaml</code>	The data behind the leaderboard site.	Harness writes, members' pull requests add
Site	GitHub Pages, rebuilt on every push to <code>main</code> .	Read-only

1.2 Data flow while driving

```
lidar, imu, encoders, camera, lap/collision counts --> bridge --> /autodrive/roboracer_1
/* --> your node
your node --> /autodrive/roboracer_1/steering_command, throttle_command --> bridge -->
simulator
```

The car takes exactly two inputs and everything else is a sensor:

- `steering_command`, `std_msgs/Float32` in $[-1, 1]$: full left to full right, ± 0.5236 rad. The actuator lags about 0.25 s and slews at 3.2 rad/s.
- `throttle_command`, `std_msgs/Float32` in $[-1, 1]$. **Zero is a hard brake**, not coasting. A car that wants to keep moving must never send zero.

1.3 Sensors (namespace /autodrive/roboracer_1/)

Topic	Type	Notes
lidar	sensor_msgs/LaserScan	1080 beams over 270°, 0.06 to 10 m, 40 Hz
imu	sensor_msgs/Imu	orientation, angular velocity, linear acceleration
left_encoder, right_encoder	sensor_msgs/JointState	wheel angle in rad; wheel radius 0.059 m
steering, throttle	std_msgs/Float32	actuator feedback
front_camera	sensor_msgs/Image	forward camera (cone tracks)
ips, odom, tf	pose	ground truth: training and debugging only
lap_count, lap_time, last_lap_time, best_lap_time, collision_count	counters	scoring: training and debugging only
reset_command	command	respawn: training and debugging only

Wheelbase 0.324 m, half width 0.135 m, steady-state speed about 23 m/s per unit throttle. With a window open the bridge exchanges data at about 10 Hz (each exchange reads the cameras back from the GPU); headless it reaches 20 to 40 Hz. Scored runs go through a proxy fixed at 10 Hz for everyone, so a driver must cope with 10 Hz. The starter kit does.

2 Rules

These follow the RoboRacer Sim Racing League.

- One attempt is 1 warm-up lap plus 10 timed laps. Total time is the sum of the timed laps plus **10 s per collision**.
- Not finishing within **300 s** is DNF, ranked after every finisher by laps completed. More than **10 collisions** is DSQ.
- Each team gets the number of attempts stated in the event; the best counts.
- **Restricted topics at race time:** ips, odom, tf, the lap and collision counters, and reset_command. The harness records which nodes subscribe to them during the run and flags the result on the site. Use them freely while developing.
- Everything a submission needs starts from the container's entrypoint (/home/autodrive_devkit.sh), never from .bashrc.
- Head-to-head: car-to-car contact counts as a collision for both cars and respawns them side by side. Finishing order is most laps first, then the finish time including penalties.

Part II

New member: from nothing to a leaderboard entry

3 Install (once, about 15 minutes)

3.1 Prerequisites

OS	Install
macOS	Docker Desktop (https://docker.com), Python 3 (comes with Xcode command line tools: <code>xcode-select -install</code>), GitHub CLI (<code>brew install gh</code>)
Windows	Docker Desktop with the WSL 2 backend, Python 3 from python.org , GitHub CLI (<code>winget install GitHub.cli</code>). Run the commands below in Git Bash or WSL.
Linux	<code>sudo apt install docker.io python3 gh</code> , then add yourself to the <code>docker</code> group and log in again

Start Docker Desktop before anything else. Then log the GitHub CLI in once: `gh auth login`.

3.2 Get the repo and the simulator

```
git clone https://github.com/Pranmani/feb-racing
cd feb-racing
./feb-sim setup --team "Your Team"
```

`setup` checks Docker, pulls the devkit image, and downloads the simulator app for your OS into `~/.feb-sim/app/`. The team name is what your results are posted under; use the name your team registers in `submissions.yaml`.

3.3 First drive: manual

```
./feb-sim run
```

A window opens on the *FEB Loop* track in Driver's Eye view. Nothing drives by itself: with no stack given, the launcher starts no driver and no opponent. Arrow keys steer and throttle. The menu in the top-left has the Manual/Autonomous button, the Track button (cycles the built-in tracks), the Cars button (more cars on the grid) and the camera choices.

macOS, if the app does not open ("damaged" or "cannot be opened"): the app is not signed by Apple. Run once:

```
xattr -dr com.apple.quarantine ~/.feb-sim/app/mac/"FEB Simulator.app"
codesign --force --deep -s - ~/.feb-sim/app/mac/"FEB Simulator.app"
```

4 Write your driver

4.1 Where the code lives

Copy the starter kit into your own folder. `stack/` is yours and is ignored by git in this repo, so nothing you do there conflicts with updates.

```
cp -r starter_kit/feb_driver stack/
```

stack/feb_driver/	
feb_driver/reactive_driver.py	THE DRIVER. Subscribes to lidar + encoders,
publishes steering + throttle	
config/driver.yaml	its tuning knobs (speeds, gap bubble, filters,
throttle law)	
launch/drive.launch.py	starts the driver with the yaml
feb_driver/sysid_probe.py	stage 2: drives test inputs and records a bag
tools/sysid_fit.py	stage 2: fits the recorded bag
package.xml, setup.py	standard ROS 2 Python package files

`reactive_driver.py` is about 170 lines. Per lidar scan it cleans the scan, zeroes a safety bubble around the nearest return, aims into the widest gap, picks a speed from the room ahead and the steering angle, and turns that into a throttle as feed-forward plus a bounded trim. Everything is commented. `driver.yaml` exposes every number.

You can add more packages next to `feb_driver`, more nodes, C++ packages, anything that builds with `colcon`. The container builds everything under `stack/` on start.

4.2 Run it on the simulator

```
./feb-sim run --stack "ros2 launch feb_driver drive.launch.py"
```

What happens, in order: the launcher starts the devkit container with `stack/` mounted inside it; the container builds your packages and starts the bridge; the simulator starts, connects to the bridge, and is put in Autonomous mode; the container starts your launch command. The car drives.

Edit, save, run the same command again. Each run is a fresh build (a few seconds for Python).

4.3 Useful options

Option	Effect
<code>-track porto</code>	another built-in track (<code>loop</code> , <code>loop_cones</code> , <code>porto</code>), or a path to any track folder
<code>-mode manual</code>	keep the keyboard even with a stack running
<code>-opponent normal</code>	the house racer in car two (<code>slow 1.5</code> , <code>normal 2.0</code> , <code>fast 2.5 m/s</code> , or <code>map</code>). Needs <code>-stack</code> . Not on Porto (single-car track).
<code>-cars 2</code>	a second, undriven car on the grid
<code>-noise 1</code>	lidar range noise and dropouts; <code>-noise "range_sigma:=0.05 latency:=0.05"</code> sets them
<code>-lidar-hz 10</code>	slower lidar, as a real bridge often delivers
<code>-camera Trackcam</code>	start in another view ("God's Eye", <code>Trackcam</code> , "Driver's Eye")

Car one is always *your* car: the Manual/Autonomous button, the keyboard, the Driver's Eye camera and topic namespace `roboracer_1` all belong to it. Your container never sees any other car.

4.4 Debug

```
./feb-sim shell          # a ROS 2 shell inside the running container
ros2 node list
ros2 topic hz /autodrive/roboracer_1/lidar
ros2 topic echo /autodrive/roboracer_1/throttle_command
rqt
./feb-sim stop           # stop the container(s) and the simulator
```

The container log (build errors, your node's output) is `docker logs feb-devkit`.

5 Score yourself and post it

```
./feb-sim practice --stack "ros2 launch feb_driver drive.launch.py" --track loop
./feb-sim submit
```

`practice` runs one attempt under competition rules: warm-up plus 10 timed laps, 10 s per collision, DNF after 300 s, through the 10 Hz proxy. It prints the result and saves `runs/<id>/` with `result.json`, logs and a ROS bag.

`submit` opens a pull request with the result file. A check validates it and merges it, and you appear on that track's practice board as *unverified*. The organiser's nightly rerun of your image turns it *verified*.

6 Race the house racer

```
./feb-sim run --stack "ros2 launch feb_driver drive.launch.py" --opponent normal
```

The house racer starts in car two only when your stack starts your car. It is the smooth lidar driver at a conservative pace with zero wall contact: the thing a new stack has to overtake. It follows a car ahead rather than ramming it.

7 Enter competitions

Competitions run on the organiser's PC from a Docker image you publish. Build it from the starter Dockerfile, which starts from the devkit image, copies `stack/` in, builds it and sets the launch command:

```
docker build -t <dockerhub-user>/feb-racer:v1 -f starter_kit/Dockerfile stack/
docker push <dockerhub-user>/feb-racer:v1
```

If your launch command differs from the starter's, change the `FEB_LAUNCH` line in a copy of the Dockerfile. Then register: add your team, GitHub logins and image to `submissions.yaml` in a pull request:

```
teams:
- name: Your Team
  members: [github-login-1, github-login-2]
  image: docker.io/<dockerhub-user>/feb-racer:v1
```

Competitions and nightly reruns pull that image. Update the tag and the yaml whenever you want a new version raced.

8 Learning path

1. **Reactive driving.** Tune `driver.yaml`. Target: 10 clean laps on loop under 100 s.
2. **System identification.** `ros2 launch feb_driver sysid.launch.py` records throttle steps, a coast and a chirp; `python3 tools/sysid_fit.py stack/sysid_bag` inside the container fits the gain and lags.
3. **Localisation and a map.** Build a map from lidar, localise on it. Check against `ips` while developing; never subscribe to it at race time.
4. **Raceline and controller.** A minimum-curvature line with a speed profile, tracked by pure pursuit, then MAP or MPC. Lap times halve here.
5. **Realism.** `-noise`, `-lidar-hz 10`, `-opponent`, the cone track.

9 If something goes wrong

Symptom	Cause and fix
Nothing drives	Correct without <code>-stack</code> . With a stack: the build failed or the node crashed, see <code>docker logs feb-devkit</code> ; or the mode button says Manual.
Port 4567 in use	Another bridge container is running: <code>./feb-sim stop</code> .
Car surges and stops	Throttle reaching zero (a hard brake). Keep feed-forward plus a bounded trim; see the starter's throttle law.
Car weaves at full lock	Raw beam-to-beam targets at 10 Hz. Low-pass and rate-limit steering; the starter shows how.
Docker pull denied	Docker Desktop not running, or the image is private; ask the organiser.
Mac app will not open	See the <code>xattr</code> and <code>codesign</code> commands in the install section.

Part III

Organiser

10 The organiser PC

Everything official runs on the team PC with the headless Linux simulator. Members never need access to it. The organiser works on it through Claude Code sessions in `tmux`; the commands below are what those sessions run.

Item	Path
Repos	<code>~/FEB/feb-racing</code> (main, this manual), <code>~/FEB/feb-sim</code> (Unity fork, branch <code>feb</code>)
Simulator players	<code>~/FEB/feb-sim/Builds/{linux,mac,windows}</code> ; Linux also linked at <code>~/feb-sim/app/linux</code>
Unity editor	<code>~/Unity/Hub/Editor/2022.3.52f1</code> , licensed through Hub sign-in
Run folders	<code>~/FEB/feb-racing/runs/<id>/</code> (<code>result.json</code> , <code>sim.log</code> , <code>container.log</code> , <code>bag</code>)
Secret tracks	<code>~/feb-secret-tracks/<name>/</code> , same layout as <code>tracks/</code> , never committed until released
Ports	4567 member devkit; 4568+ harness containers, one per car; 4580 harness proxy; 4565/4566 launcher proxy and opponent

11 Tracks

A track is a folder. The simulator reads `track.json` at launch, so adding a track never needs a Unity rebuild; it needs an app release only to ship inside the app.

```
tracks/<name>/
  design.yaml  optional: waypoints + width, rendered to map.png
  map.png      white = drivable corridor, everything else = not drivable
  track.yaml   name, resolution, direction, checkpoints, walls, cones, max_cars
  track.json   GENERATED: walls, centreline, checkpoints, spawn, cones
  preview.png  GENERATED: picture for the site
```

11.1 Create a new track (two commands)

```
mkdir tracks/hairpins
```

```

cat > tracks/hairpins/design.yaml <<EOF
width: 2.0
points: [[0, 0], [12, 0], [12, 4], [4, 4], [4, 8], [14, 8], [14, 12], [0, 12]]
EOF
cat > tracks/hairpins/track.yaml <<EOF
name: Hairpins
direction: ccw
checkpoints: 20
walls: {diameter: 0.33}
EOF
tools/track_design.py tracks/hairpins/design.yaml # map.png, resolution and origin
    into track.yaml
tools/track_build.py tracks/hairpins               # track.json + preview.png
./feb-sim run --track hairpins                     # look at it

```

The corridor is a smooth closed spline through the waypoints: any number, any shape, no self-intersections. Any other PNG works too: hand-painted, or a SLAM occupancy grid of a real track (Porto is one; its resolution and origin come from the ROS `map.yaml`).

track.yaml key	Meaning
name	shown in the simulator and on the site
direction	ccw or cw
checkpoints	lap checkpoints; checkpoint 0 is the finish line
max_cars	1 for corridors too narrow to race side by side (Porto); default 4
start	optional [x, y] finish-line position; default: the straightest section
walls.diameter	air-duct diameter, 0.33 m like the league; omit for a cone-only track
cones.spacing	metres between cones on each edge, blue left, yellow right; omit for none

11.2 Secret tracks

Design competition tracks in `~/feb-secret-tracks/<name>/` with the same two commands. The event file points at that folder and the harness passes it to the headless simulator directly. After the event, move the folder into `tracks/`, commit, and cut an app release so it ships inside the simulator as a practice track.

12 Events

One YAML per event in `events/`. Pushing it lists the event on the site.

```

id: 2026-10-qualification
title: Fall 2026 Qualification
date: 2026-10-15
mode: time-attack           # or head-to-head
status: upcoming           # upcoming | live | done
track: ~/feb-secret-tracks/qual-2026-10
attempts: 2                 # best attempt counts
rules: {laps: 10, warmup_laps: 1, collision_penalty_s: 10, max_collisions: 10, timeout_s: 300}
entries: all                # or a list of team names

```

13 Race day: time-attack

```

cd ~/FEB/feb-racing
git pull                               # latest submissions.yaml
./feb-race event 2026-10-qualification # every registered image, N attempts each
git add results && git commit -m "qualification results" && git push

```

Each attempt: pull the image, start the container, start the headless simulator on the secret track behind the 10 Hz proxy, probe laps and collisions inside the container, record a bag, tear down, write `result.json`. Standings print at the end and the site rebuilds itself on push. Afterwards set `status: done` in the event file and release the track.

One attempt takes about 3 minutes. Ten teams with two attempts is about an hour. `-gui` on any harness command shows the simulator window instead of running headless.

14 Finals: head-to-head

```
./feb-race bracket 2026-12-final --seed-from 2026-10-qualification
./feb-race h2h --teams "Team A" "Team B" --track tracks/loop --laps 10 --gui # one
race
```

Seeds come from the time-attack standings: single elimination, 1 v N, 2 v N-1, top seed has a bye on an odd count. Each car runs in its own container behind the proxy, so every container sees its own car as `roboracer_1`, unchanged from time-attack. Results carry the race id and position; the site shows the bracket. Two cars is the supported race mode; three or more works end to end but drivers do not see each other, so treat it as experimental.

15 Between events

```
./feb-race nightly # rerun every registered image on
every practice track
./feb-race verify results/practice/loop/<file>.json # rerun one self-reported entry
./feb-race standings 2026-10-qualification
./feb-race site # rebuild site/data.json locally
```

Put `nightly` in cron on the PC and push `results/` afterwards. Self-reported practice results arrive as pull requests that validate and merge themselves; the `nightly` turns them verified.

16 Releasing the simulator

Needed only when the app itself changes or a released track should ship inside it.

```
cd ~/FEB/feb-sim
./compile_check.sh # compiles Assets/FEB, no licence needed
./build.sh scene && ./build.sh linux && ./build.sh mac && ./build.sh windows
tar -C Builds/linux -czf FEB-Simulator-linux.tar.gz .
(cd Builds/mac && zip -qr ../../FEB-Simulator-mac.zip "FEB Simulator.app")
(cd Builds/windows && zip -qr ../../FEB-Simulator-windows.zip .)
gh release upload v0.1.0 -R Pranman1/feb-racing --clobber FEB-Simulator-*.tar.gz FEB-
Simulator-*.zip
```

`feb-sim setup` downloads from the latest release. Build details and what the fork adds are in `docs/build.md`. The RoboRacer vehicle prefab and the upstream scripts stay untouched; everything FEB-specific is under `Assets/FEB`.

17 A semester

When	What	Where it shows
Week 0	Install, drive manually, run the starter driver, post one practice result	loop practice board
Weeks 1 to 5	Self-paced learning path, one practice track per stage	practice boards, nightly reruns
Mid-semester	Qualification: time-attack on a secret track, 2 attempts	the event page
Weeks 7 to 12	Weekly open rounds on rotating tracks; released tracks join the pool	practice boards
Finals	Time-attack on a new secret track for seeding, then a head-to-head knockout	the event page with races

Roles: the organiser owns the secret tracks, the events folder and the PC; members own their stack/, their images and their practice results.

18 Organiser checklist for the first run-through

1. Make the devkit package public on GitHub (Packages, `feb-devkit`, Package settings, Change visibility). Until then `feb-sim setup` fails at the pull for anyone but you.
2. On a member laptop (your Mac): follow Part II exactly as written, from `git clone` to `./feb-sim submit`. Note every step that needs a hint the manual does not give.
3. Design the first secret track, write its event file, push it.
4. Run `./feb-race event` once with the example team to see a full result land on the site.

A Command reference

Command	What it does
<code>feb-sim setup -team T</code>	pull the devkit image, download the app, set the team name
<code>feb-sim run [-track] [-stack] [-mode] [-opponent] [-cars] [-noise] [-lidar-hz] [-camera] [-headless]</code>	start the container and the simulator, connected
<code>feb-sim practice -stack S [-track] [-laps] [-headless]</code>	one scored attempt into <code>runs/<id>/</code>
<code>feb-sim submit [-run DIR]</code>	pull request with the newest (or given) result
<code>feb-sim shell</code>	ROS 2 shell inside the running container
<code>feb-sim stop</code>	stop containers and the simulator
<code>feb-race event ID</code>	run a time-attack event
<code>feb-race bracket ID -seed-from ID</code>	run a head-to-head knockout
<code>feb-race h2h -teams A B -track T</code>	one race
<code>feb-race run -image I -team T -track T</code>	one attempt of one image
<code>feb-race nightly, standings ID, site</code>	reruns, verification, standings, site data

B Repository layout

```
feb-racing/  
  feb-sim, feb-race      launchers (Python 3, no dependencies beyond docker and gh  
  )  
  febrace/              harness library: runner, probe, proxy, rules, results,  
  site, h2h  
  devkit/              Dockerfile + entrypoint of the devkit image, feb_tools (  
  house driver, noise)  
  starter_kit/          feb_driver package + submission Dockerfile  
  stack/               YOUR code (gitignored)  
  tracks/              public tracks  
  events/, results/, submissions.yaml, site/    leaderboard data and page  
  docs/               handbook and detail pages; this manual in docs/manual  
  runs/              local attempt folders (gitignored)  
feb-sim/ (branch feb)  Unity fork; Assets/FEB holds everything FEB-specific
```